

production app — file-by-file GUIDE

Business view: [config/production/README.md](#). Sabse bada app: Adda lifecycle, stages, worker truth, costing freeze.

models/ (split by domain — har file = ek concern)

File	What lives here	Pattern / why
<code>core.py</code>	Product, Stage (library), WorkflowStage (+RoleRate)	WorkflowStage = per-product POLICY row (order, cost_rate dual-duty ADR-0009, credits_workers, cost_billed_at grouping; TM-1 field aayega yahin)
<code>adda.py</code>	Adda, AddaStageRecord (+ pending_report_workers helper)	SR = stage ka polymorphic parent; processing_cost frozen (honest-NULL)
<code>worker_task.py</code>	WorkerStageTask, WorkerStageContribution	THE production truth (ADR-0005, C-TM); partial-unique active task; settlement_line provenance string-FK
<code>layering.py</code>	LayeringRecord, LayeringRollEntry, RemainingClothOfClothRoll	per-roll verify + MANDATORY left-overs (G1 ke facts)
<code>cutting.py</code>	CuttingRecord, ProductSize, PatternVerification, PieceBreakup, Bundle(+items), ProductPattern	cutting = first real quantities; breakup = expected denominators
<code>barcode.py</code>	BarcodeGenerationRecord	archetype-E stage record

services/ (ALL writes — views kabhi nahi)

File	Role
<code>worker_task_service.py</code>	★ chokepoint: WST/WSC sole writer, expected freeze, verified qty (P1), auto-cancel resolve
<code>adda_service.py</code>	Adda create (race-safe per-product counter) + stage advance
<code>cost_service.py</code>	processing_cost freeze/clear; <code>role_rate_for</code> (grouped-member guard C-1)
<code>flow_service.py</code>	WorkflowStage CRUD + grouping guards
<code>_shared.py</code>	auth helpers + ★ reopen_stage_record skeleton (V2-3 settled-block)
<code>access_service.py</code>	skill-gating reads
<code>activity_service.py</code>	timeline UNION reads
<code>product_service.py</code> / <code>product_size_service.py</code>	masters
<code>reconciliation_service.py</code>	counter sanity (read-only)

stages/ — the OPEN-CLOSED engine

`base/handler.py` (contract: typed record, complete validations, cost_quantity, contribution_schema) + `base/registry` + per-stage packages (layering/cutting/cutting_pattern/barcode_generation each = handler + service). NAYA STAGE = naya package; worker UI ko haath nahi lagana padta.

views/ (12 modules — parse → gate → delegate ONLY)

FILE MAPs already in code: `stage_views.py` (1.4k — layering+cutting consoles, 6 sections documented at top), `pattern_stage_views.py`, `barcode_gen_views.py`, `worker_report_views.py` (★ phone report + AddaReportReviewView P1), `adda_views.py`, `flow_views.py`, `costing_views.py`, `dashboard.py`, `product_views.py`, `pattern_views.py`, `access_views.py`, `mixins.py` (RBAC).

forms/ · urls.py · templates/

forms = plain Django forms per stage (`_shared.py` = worker chips widget). urls.py header me poora route-group map. Templates: `_stage_panel_*.html` (operator consoles), `worker_report*.html` (phone), `product_flow.html` (editor).

Dots kaise connect (ek request)

```
worker report POST → urls → WorkerReportView ( assignment gate )
  → worker_task_service.report_contributions/complete (WSC + freeze)
  → history via stage services · settlement (expense) baad me READS WSC
```

Topics yahan use hote hain — kahan padhein

Har concept ka official link + “is project me kahan” mapping:

[../LEARNING/10 ONLINE RESOURCES.md](#). App ka business-view: README (code ke saath). Deep lessons: [docs/LEARNING/](#).